



PRODUCTION FOUNDATIONS

Linux, SSH, Docker, Nginx and the Workflows Around Them

A hands-on path from your first Ubuntu shell to a small, defensible production deployment.



Includes terminal skills • Nano and Vim • networking • virtual machines • SSH keys • systemd • logs • Docker and Compose • reverse proxies • HTTPS • Git • troubleshooting • capstone

Prepared as a practical learning document • July 2026

How to use this guide

This guide assumes you can use Windows, macOS, or Linux as your main computer. You will create an Ubuntu Server virtual machine and treat it like a remote server. That gives you a disposable place to make mistakes, reset snapshots, and repeat commands.

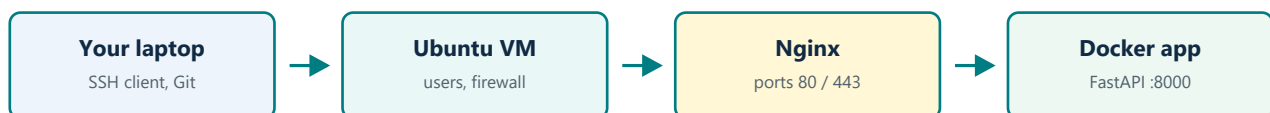
Read one module, type the commands yourself, and keep a lab notebook. Record the command, the result, and the reason it worked. Copying commands without inspecting their output builds fragile confidence.

Safety rule

Run destructive commands only inside the practice VM and verify your current directory first. Treat `rm -rf`, disk partitioning, firewall changes, and SSH authentication changes as operations that need a rollback plan.

The mental model

A production web service is a chain of boundaries. Your laptop crosses the network through SSH. The server kernel accepts traffic on selected ports. Nginx receives HTTP requests. Docker isolates the application process and its dependencies. Logs tell you which boundary failed.



Browser path client → DNS → VM public IP → Nginx → private app port

Admin path terminal → SSH :22 → shell → systemctl / docker / logs

The database can live in another container or a managed service. Do not expose it to the public Internet.

Figure 1. The full path you will build and troubleshoot.

Learning loop

Step	What you do	What you prove
1. Predict	Say what the command should change.	You understand intent.
2. Execute	Run one small change.	You can isolate cause and effect.
3. Inspect	Read status, logs, ports, files, and exit codes.	You have evidence.
4. Break	Change one setting or stop one service.	You can recognize failure.
5. Repair	Restore the service without rebuilding everything.	You can operate it.

Table of contents

How to use this guide	2
The mental model	2
Learning loop	2
Table of contents	3
What “industry standard” means here	7
Start here: curl and the command-line toolbox	8
Why curl belongs near the beginning	8
The request path you are testing	8
curl patterns you will reuse	9
Diagnose by layer	9
The command-line toolbox	10
Editors: Nano and Vim	10
FFmpeg and ffprobe	11
jq, ripgrep, and tmux	11
Lab: inspect, transform, verify	11
Cron jobs and scheduler choices	12
The five schedule fields	12
Write the environment into the schedule	12
Design the job for retries and overlap	12
systemd timers on one Linux host	13
What people mean by a cron manager	13
Heartbeat monitoring	13
1 The terminal, shell, and filesystem	14
Terminal versus shell	14
The filesystem hierarchy	14

Create, copy, move, and remove	14
Read and search	15
Redirection and pipes	15
Lab: Build a command-line evidence folder	15
2 Users, permissions, packages, and processes	16
Identity and least privilege	16
Reading mode strings	16
Packages on Ubuntu	16
Processes and signals	16
Lab: Trace a process to a port	17
3 Editing on a server: Nano and Vim	18
Choose the tool that preserves accuracy	18
Nano	18
Vim's modal model	18
Edit configuration defensibly	18
Lab: Make the same edit twice	19
4 Networking without hand-waving	20
The layers you need every day	20
Addresses and bind scope	20
DNS and HTTP inspection	20
A boundary-first diagnosis	20
Common ports	21
Lab: Prove each hop	21
5 Virtual machines as a safe server lab	22
What a VM isolates	22
VM versus container	22
Local hypervisor choices	22
Network modes	22
Suggested Ubuntu VM	23
Snapshots are checkpoints, not backups	23
Lab: Prepare the Ubuntu server	23
6 SSH keys and remote administration	24
What SSH protects	24
Public and private keys	24
Generate and install a key	24
A useful client configuration	24
Agent and forwarding	24
File transfer and persistent work	25
Harden without losing access	25
Lab: Key-only SSH with a tunnel	26
7 Services, systemd, and logs	27
Service lifecycle	27

Read the journal with a question	27
A minimal custom service	27
Exit state is evidence	28
Lab: Create and repair a service	28
8 Docker: images, containers, storage, and networks	29
The container mental model	29
Daily commands	29
Ports: publish is a host-level decision	29
Storage choices	29
Container networks	30
Debug from both sides	30
Lab: Run, inspect, and replace a container	30
9 Dockerfiles and Compose	31
Dockerfile versus Compose	31
A production-minded Python image	31
Compose for one application	32
Operate the Compose project	32
Tags, digests, and releases	32
Lab: Build a health-checked service	33
10 Nginx and the reverse proxy boundary	34
Can an app run without Nginx?	34
Forward proxy versus reverse proxy	34
Request flow	34
Minimal reverse proxy	34
Enable, validate, reload, verify	35
Static files and uploads	35
Load balancing is one capability, not magic scaling	35
Lab: Put Nginx in front of a private backend	35
11 TLS, firewalls, secrets, and server hardening	36
Four different security questions	36
Host firewall with UFW	36
HTTPS and certificates	36
Secrets	36
Patch, minimize, and review	37
Backups require restore tests	37
Lab: Reduce the exposed surface	37
12 Git, deployment, observability, and incident habits	38
Git is history, not deployment	38
Configuration as code	38
A small release protocol	38
Logs, metrics, and traces answer different questions	39
Minimum useful application log fields	39

Incident loop	39
Automation progression	39
Lab: Write and rehearse a release checklist	40
13 Capstone: deploy a FastAPI service on the VM	41
Target architecture	41
Pre-flight checklist	41
1. Create the project layout	42
2. Application	42
3. Image definition	43
4. Runtime definition	44
5. Validate and start the app	44
6. Configure host Nginx	44
7. Apply and verify firewall policy	45
8. Observe the full request	45
9. Optional domain and HTTPS	45
10. Reboot and rollback tests	45
Capstone acceptance checklist	46
14 Troubleshooting playbook	47
The golden sequence	47
Fast triage	47
Failure matrix	47
Docker-specific evidence	48
Nginx-specific evidence	48
High-noise actions to resist	48
Practice fault injection	48
15 Interview explanations and a 30-day practice plan	49
Short explanations that preserve the important detail	49
Interview depth ladder	50
30-day plan	50
Portfolio evidence	50
A Command cheat sheets	51
Linux navigation and files	51
Nano and Vim	51
Networking	51
SSH	52
systemd and logs	52
Docker and Compose	52
Nginx	52
B Glossary and sources	53
Glossary	53
Primary sources	53
Suggested next subjects	54

What “industry standard” means here

Companies differ in cloud provider, orchestration platform, editor, and deployment tooling. The durable skills sit beneath those choices: least privilege, version-controlled configuration, repeatable builds, encrypted administration, explicit network boundaries, health checks, logs, backups, and a tested rollback.

This guide uses Ubuntu Server, OpenSSH, systemd, Docker Compose, Nginx, Git, and a FastAPI example. The concepts transfer to Debian, Red Hat, cloud VMs, Kubernetes, managed load balancers, and other application frameworks.

Layer	Tool in this guide	Alternatives you may meet
Host	Ubuntu Server VM	Debian, RHEL, Amazon Linux, cloud images
Remote access	OpenSSH	Cloud session managers, bastion hosts, VPN/zero-trust access
Editor	Nano and Vim	Neovim, Emacs, VS Code Remote SSH
Service manager	systemd	OpenRC, supervisord, platform-managed processes
Packaging	Docker + Compose	Podman, Kubernetes, Nomad, managed containers
Web edge	Nginx	HAProxy, Caddy, Envoy, cloud load balancer/API gateway
Certificates	Let's Encrypt / Certbot	Cloud certificate managers, internal PKI

Scope

This is a foundation course, not a promise that one small VM equals a complete production platform. High-availability systems add managed databases, secret stores, centralized logs, metrics, backups, automated delivery, redundancy, and incident procedures.

Start here: curl and the command-line toolbelt

Outcome. Use curl to inspect a real HTTP exchange, then recognize the command-line tools engineers reach for when they search, edit, transform, transfer, and diagnose data.

Why curl belongs near the beginning

curl is a command-line client for transferring data with URLs. In server work, you will use it to test an API, check a health endpoint, inspect headers, reproduce a failing request, download an artifact, and prove whether a proxy can reach a backend. The curl project supports many protocols; this guide concentrates on HTTP and HTTPS. [C1]

The name

Read curl as “client for URLs.” The command prints the response body by default. Flags decide whether you also see headers, diagnostics, status handling, or a saved file.

The request path you are testing

Stage	What curl helps you observe	Typical failure
URL parsing	scheme, host, port, path, query	malformed URL or wrong path
DNS	name resolution in verbose output	name does not resolve
Connection	target IP and TCP connection	timeout or connection refused
TLS	certificate name, chain, negotiated protocol	untrusted or mismatched certificate
HTTP request	method and outgoing headers	wrong Host, auth, or content type
HTTP response	status, headers, body, timing	401, 404, 429, 500, or proxy error
Process result	curl exit code	script can detect transfer failure

Your first requests

```
curl https://example.com
curl -i https://example.com
curl -I https://example.com
curl -v https://example.com
curl -sS https://example.com
curl -fsS https://example.com/health
```

Read the flags

Flag	Meaning	Use
-i	include response headers with the body	inspect status and content type
-I	send an HTTP HEAD request	metadata check; the server may treat HEAD differently
-v	show connection, TLS, request, and response diagnostics	interactive diagnosis
-sS	hide progress but still show errors	clean script or pipeline output
-f	return failure for HTTP 400 or later	make automation notice bad HTTP status

Body output is not success

Without `--fail` or `--fail-with-body`, curl can exit successfully after receiving an HTTP 404 or 500 because the network transfer itself succeeded.

Do not normalize -k

`-k` disables TLS certificate verification. Fix the certificate name, chain, clock, or trust store instead of hiding the failure.

curl patterns you will reuse

GET requests, headers, and query parameters

```
curl --get \
  --data-urlencode "query=reverse proxy" \
  --data-urlencode "limit=10" \
  -H "Accept: application/json" \
  https://api.example.com/search

curl -D response-headers.txt -o response-body.json \
  https://api.example.com/items/42
```

-D saves response headers from the real request. This differs from -I, which changes the method to HEAD. Use --data-urlencode when query values may contain spaces or reserved characters.

JSON requests and authentication

```
export API_TOKEN='replace-with-a-real-secret-source'

curl --fail-with-body --silent --show-error \
  --request POST \
  --header "Authorization: Bearer ${API_TOKEN}" \
  --json '{"name":"lab-service","enabled":true}' \
  https://api.example.com/services

unset API_TOKEN
```

Protect credentials

Command arguments can appear in shell history, process listings, CI logs, and support transcripts. Prefer short-lived credentials supplied by a secret manager or protected file descriptor. Redact verbose output before sharing it.

Downloads, redirects, and uploads

```
curl --fail --location --output release.tar.gz \
  https://downloads.example.com/release.tar.gz

curl --continue-at - --output release.iso \
  https://downloads.example.com/release.iso

curl --fail-with-body --form "file=@report.pdf" \
  https://api.example.com/uploads
```

Flag	Meaning
-L / --location	follow HTTP redirects; review cross-host credential behavior
-o FILE	write output to the chosen file
-O	use the remote file name; verify it before trusting the result
-C -	resume a partial download when the server supports ranges
-F / --form	send multipart form data, commonly for file upload
--max-time 10	limit total request time so automation does not hang
--retry 3	retry selected transient failures; do not hide persistent errors

Diagnose by layer

Symptom	First evidence
Could not resolve host	check spelling, DNS, VPN, and <code>getent hosts</code>
Connection timed out	route and firewall; compare target IP and port
Connection refused	host reachable, but no listener accepts that port
Certificate verify failed	name, validity, chain, clock, and trust store
401 / 403	authentication or authorization, not basic connectivity
404	host/path/routing; inspect Host header and proxy configuration
502 / 504	reverse proxy cannot reach or receive a timely backend response

The command-line toolbelt

Command-line tools work well together because many read standard input, write standard output, report errors on standard error, and return an exit code. Pipes join small operations without forcing every tool to understand every file format.

Inspect, search, and select

Tool	Job	Starter command
file	identify a file from its contents	file download.bin
less	page through text without loading it into an editor	less app.log
find	select filesystem entries by name, type, time, or size	find . -type f -name '*.conf'
grep	portable line-oriented pattern search	grep -Rni 'ERROR' logs/
rg	fast recursive project search with ignore rules	rg -n -tpy 'TODO' .
jq	filter and transform JSON	jq '.items[] .name' response.json
cut	select delimited fields	cut -d, -f1,3 data.csv
sort / uniq	order values and count duplicates	sort names.txt uniq -c

Transform, package, and transfer

Tool	Job	Starter command
sed	stream-oriented text replacement and selection	sed 's/dev/prod/g' input.txt
awk	field-based text processing and reports	awk -F, '{sum += \$3} END {print sum}' data.csv
xargs	build commands from input items	printf '%s\0' *.log xargs -0 -n1 gzip
tar	archive a directory while preserving structure	tar -czf backup.tar.gz directory/
zip / unzip	interchange ZIP archives	unzip -l archive.zip
rsync	incremental local or SSH file synchronization	rsync -av --dry-run src/ host:dst/
scp / sftp	simple file transfer over SSH	scp report.pdf host:srv/reports/
curl / wget	scriptable URL transfer and downloads	curl -fL -o file URL

Observe and control systems

Tool	Job	Starter command
ps / top / htop	process and resource inspection	ps -ef --forest
ss / lsof	sockets and the processes that own them	sudo ss -lntup
dig / getent	DNS and operating-system name resolution	getent hosts example.com
openssl	TLS, certificates, hashes, and cryptographic inspection	openssl s_client -connect host:443 -servername host
journalctl	query systemd journal entries	journalctl -fu nginx
tmux / screen	persistent terminal sessions	tmux new -s deploy
watch	repeat a read-only command at an interval	watch -n 2 docker compose ps
nc	basic TCP/UDP connectivity tests	nc -vz host 443

Editors: Nano and Vim

Nano keeps common shortcuts visible and suits quick, careful edits. Use Ctrl+O to save, Ctrl+X to exit, and Ctrl+W to search. Vim uses modes: press i to insert, Esc to return to Normal mode, :wq to save and exit, or :q! to discard changes. The full editor lab appears in Module 3. Use sudoedit for protected configuration files.

Portability versus convenience

grep, sed, and awk exist on many Unix-like systems. rg, jq, and htop may need installation. Check availability with `command -v TOOL`.

Popular tools in practice

FFmpeg and ffprobe: inspect before converting

ffprobe reports streams, codecs, dimensions, duration, and container metadata. ffmpeg reads one or more inputs, applies selected streams and filters, then writes one or more outputs. The usual shape is `ffmpeg [input options] -i input [output options] output`. [C2]

```
ffprobe -hide_banner input.mp4

# Convert to H.264 video and AAC audio
ffmpeg -i input.mov -c:v libx264 -crf 23 -c:a aac output.mp4

# Extract audio
ffmpeg -i input.mp4 -vn -c:a libmp3lame -q:a 2 audio.mp3

# Scale to 1280 pixels wide while preserving aspect ratio
ffmpeg -i input.mp4 -vf "scale=1280:-2" -c:a copy output-720p.mp4

# Copy streams without re-encoding; fast, but cuts may align to keyframes
ffmpeg -ss 00:01:00 -i input.mp4 -t 30 -c copy clip.mp4
```

Container is not codec

MP4 and MKV are containers. H.264, AV1, AAC, and Opus are codecs. Changing the file extension does not transcode media. Inspect with ffprobe and verify the output duration, streams, resolution, and playback.

jq and ripgrep

```
curl -fsS https://api.example.com/items \
  | jq -r '.items[] | select(.enabled) | [.id, .name] | @tsv'

rg -n -S "proxy_pass" /etc/nginx
rg -n -tpy "except Exception" .
rg --files | rg '\.(yaml|yml)$'
```

jq parses JSON and emits valid transformed output; it is safer than grepping structured JSON. [C3] ripgrep searches directories recursively and respects common ignore files by default. Use `rg -uuu` only when you intend to include ignored, hidden, and binary files. [C4]

tmux: keep remote work alive

```
tmux new -s deploy
# Detach: Ctrl+B, then D
tmux list-sessions
tmux attach -t deploy
tmux kill-session -t deploy
```

Lab: inspect. transform. verify

```
mkdir -p ~/labs/toolbelt && cd ~/labs/toolbelt

curl -fsSL -o sample.json https://api.github.com/repos/curl/curl
file sample.json
jq '{name, language, open_issues_count}' sample.json | tee summary.json
rg -n 'language|open_issues' summary.json
tar -czf evidence.tar.gz sample.json summary.json
tar -tzf evidence.tar.gz
sha256sum evidence.tar.gz
```

- curl exits successfully and writes valid JSON.
- jq parses the file and produces the chosen fields.
- rg finds the expected keys.
- tar lists both source files, and the checksum gives you a stable verification value.

Official references

[C1] curl documentation • [C2] FFmpeg documentation • [C3] jq manual • [C4] ripgrep user guide

Cron jobs: schedule small recurring work

cron runs commands from schedule tables called crontabs. It suits small host-local jobs such as rotating a custom file, generating a report, calling a health endpoint, or running a backup wrapper. Each user can own a crontab; system crontabs can specify which user runs the command. [C5]

The five schedule fields

Minute	Hour	Day of month	Month	Day of week	Command
0-59	0-23	1-31	1-12	0-7	/absolute/path/to/job

Expression	Runs
*/5 * * * *	every five minutes
15 2 * * *	daily at 02:15
0 9 * * 1-5	09:00 Monday through Friday
0 0 1 * *	midnight on the first day of each month
@reboot	after the cron daemon starts; implementation support varies

Day fields can surprise you

In common Vixie-style cron implementations, if both day-of-month and day-of-week are restricted, a match in either field can trigger the job. Test expressions against the cron implementation you run.

Edit and inspect your crontab

```
crontab -l          # list your jobs
crontab -e          # edit with $VISUAL or $EDITOR
systemctl status cron --no-pager
journalctl --since "1 hour ago" -t CRON

# Avoid crontab -r until you intend to remove every job.
```

Write the environment into the schedule

```
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
MAILTO=""

# At 02:15 UTC, skip if the previous run still owns the lock.
15 2 * * * /usr/bin/flock -n /tmp/daily-report.lock \
/home/labuser/bin/daily-report.sh \
>> /home/labuser/logs/daily-report.log 2>&1
```

- Use absolute executable and file paths. Cron starts with a small environment and no interactive shell profile.
- Create the log directory before the job runs and configure retention.
- Set the intended timezone at the host or scheduler level and test daylight-saving transitions.
- Escape percent signs in a crontab command when your cron implementation treats % specially.
- Run the script manually as the scheduled user before installing the schedule.

Design the job for retries and overlap

A scheduler can start the same job twice after retries, clock changes, failover, or delayed delivery. Make the work idempotent, use a lock or concurrency policy, write outputs atomically, set a timeout, and record a run ID plus start, success, failure, and duration.

Cron does not guarantee completion

Cron starts a command. It does not give you dependencies, a durable run history, distributed locking, backfill, or an alert when the whole machine is down. Add those capabilities or choose a stronger scheduler.

Cron managers, timers, and workflow schedulers

systemd timers on one Linux host

A systemd timer activates a service unit. You gain journal logs, service users, resource controls, dependencies, randomized delay, and optional catch-up after downtime through `Persistent=true`. [C6]

```
# /etc/systemd/system/daily-backup.service
[Service]
Type=oneshot
User=labuser
ExecStart=/home/labuser/bin/backup.sh
NoNewPrivileges=true

# /etc/systemd/system/daily-backup.timer
[Timer]
OnCalendar=*-*-* 02:15:00
Persistent=true
RandomizedDelaySec=10m

[Install]
WantedBy=timers.target

sudo systemctl daemon-reload
sudo systemctl enable --now daily-backup.timer
systemctl list-timers --all
systemctl status daily-backup.timer --no-pager
journalctl -u daily-backup.service -n 100 --no-pager
sudo systemctl start daily-backup.service # manual test
```

What people mean by a “cron manager”

Category	Use it when	Examples and key controls
Cron or systemd UI	operators need a central inventory and controlled manual runs	self-hosted operations consoles; require RBAC, audit logs, and secret handling
Heartbeat monitor	an existing job needs missed-run and duration alerts	Healthchecks.io or Cronitor; jobs send start/success/failure pings [C10]
Kubernetes CronJob	the workload already runs in a cluster	concurrencyPolicy, deadlines, time zone, job history, idempotent tasks [C7]
Cloud scheduler	a managed service should invoke an API, queue, function, or cloud job	EventBridge Scheduler or Cloud Scheduler; identity, retries, DLQ, and logs [C8]
Workflow orchestrator	tasks have dependencies, retries, backfills, data intervals, or many owners	Airflow, Prefect, or Dagster; DAG/run history, workers, pools, and alerts [C9]

Heartbeat monitoring

A heartbeat monitor acts as a dead-man's switch. The job sends a success request after it finishes. The monitor alerts when the ping arrives late or never arrives. Send start and failure events when you also need duration and explicit failure visibility.

```
if /home/labuser/bin/backup.sh; then
  curl -fsS --max-time 10 --retry 3 -o /dev/null \
    "https://monitor.example/ping/JOB_TOKEN"
else
  status=$?
  curl -fsS --max-time 10 -o /dev/null \
    "https://monitor.example/ping/JOB_TOKEN/fail" || true
  exit "$status"
fi
```

Choose by failure model

Use cron for one simple host-local command. Use a systemd timer when Linux service controls and catch-up matter. Use a platform scheduler where the workload runs. Use an orchestrator when dependencies and backfills become part of the problem.

Official references

[C5] Ubuntu crontab manual • [C6] systemd.timer • [C7] Kubernetes CronJob • [C8] EventBridge Scheduler • [C9] Airflow scheduler • [C10] Healthchecks.io

1 The terminal, shell, and filesystem

Outcome. Navigate a Linux server, inspect state, manipulate files safely, and understand where configuration lives.

Terminal versus shell

The terminal is the window that carries text input and output. The shell is the program interpreting your commands. Ubuntu commonly gives interactive users Bash. The prompt contains context; it does not form part of the command.

```
whoami          # current user
hostname        # machine name
pwd             # current working directory
echo "$SHELL"   # configured login shell
uname -a        # kernel and machine information
cat /etc/os-release # Linux distribution and release
```

The filesystem hierarchy

Path	Purpose	Typical example
/home/	Personal files and per-user configuration	~/ssh/config
/etc	System-wide configuration	/etc/nginx/nginx.conf
/var	Changing service data, logs, caches	/var/log/nginx/
/usr	Installed programs and shared read-only data	/usr/bin/python3
/opt	Optional or self-contained application software	/opt/myapp
/srv	Data served by the machine	/srv/www
/tmp	Temporary files; cleanup policy varies	/tmp/build-output
/proc, /sys	Kernel and process views, not ordinary disk files	/proc/cpuinfo

Paths

An absolute path starts at /. A relative path starts at your current directory. ~ expands to your home directory. . means the current directory and .. means its parent.

```
cd /var/log      # absolute
cd nginx         # relative: /var/log/nginx
cd ~             # your home
cd -             # previous directory
ls -lah          # long list, hidden files, readable sizes
find . -maxdepth 2 -type f -name "*.conf"
du -sh .         # size of current tree
df -h           # mounted filesystem capacity
```

Create, copy, move, and remove

```
mkdir -p ~/labs/terminal
cd ~/labs/terminal
touch notes.txt
cp notes.txt notes.backup.txt
mv notes.txt commands.txt
mkdir archive
mv notes.backup.txt archive/
rm archive/notes.backup.txt
rmdir archive
```

Before removal

Run `pwd` and `ls -la -- target`. Use an explicit target. A shell expands wildcards before `rm` receives them.

Read and search

```
cat file.txt           # short file
less /var/log/syslog   # paged view; q exits
head -n 20 file.txt
tail -n 50 file.txt
tail -f /var/log/nginx/access.log
grep -Rni "listen 80" /etc/nginx
command -v nginx
man ssh                # manual; / searches, q exits
```

Redirection and pipes

A program has standard input, standard output, and standard error. A pipe sends one program's standard output into another program's standard input. Redirection sends output to a file.

```
printf "alpha\nbeta\nalpha\n" > names.txt  # overwrite
printf "gamma\n" >> names.txt              # append
sort names.txt | uniq -c
grep "ERROR" app.log > errors.txt
some-command > output.log 2> error.log
some-command > combined.log 2>&1
```

Lab: Build a command-line evidence folder

Goal

Create a small inventory of the practice VM without changing system configuration.

```
mkdir -p ~/labs/inventory
cd ~/labs/inventory
date -Is > captured-at.txt
uname -a > kernel.txt
cat /etc/os-release > os-release.txt
df -h > filesystems.txt
ip -brief address > addresses.txt
ss -ltnup > listening-ports.txt
find . -maxdepth 1 -type f -printf "%f\n" | sort
```

Evidence of success

- Every command returns exit code 0: run `echo $?` after one.
- The directory contains six timestamped or state-capture files.
- You can explain which files describe storage, networking, and operating system state.

2 Users, permissions, packages, and processes

Outcome. Explain ownership and modes, use sudo deliberately, install software, and inspect running programs.

Identity and least privilege

Linux evaluates access through user IDs, group IDs, ownership, and permission bits. A normal account should perform daily work. sudo runs one authorized command with elevated privileges and leaves an audit trail. Do not stay in a root shell for convenience.

```
id
groups
getent passwd "$USER"
sudo -l
sudo whoami      # should print root
whoami           # still your normal user
```

Reading mode strings

In -rwxr-x---, the first character describes the file type. The next triples grant read, write, and execute to the owner, group, and everyone else. Directories use execute permission to allow traversal.

Numeric	Symbolic	Meaning
7	rwx	read + write + execute
6	rw-	read + write
5	r-x	read + execute
4	r--	read only
0	---	no access

```
ls -l deploy.sh
chmod u+x deploy.sh
chmod 750 deploy.sh
chmod 600 ~/.ssh/id_ed25519
sudo chown www-data:www-data /srv/www
namei -l /srv/www/index.html      # permissions across the entire path
```

Avoid chmod 777

World-writable files trade a permissions error for a security problem. Identify the process user, choose the correct owner/group, and grant only the required mode.

Packages on Ubuntu

apt resolves packages from configured repositories. Refresh repository metadata before installing. Upgrades and third-party repositories affect the whole machine, so read the proposed changes.

```
sudo apt update
apt list --upgradable
apt show nginx
sudo apt install nginx
dpkg -L nginx | less
apt-cache policy nginx
sudo apt remove nginx      # leaves some configuration
sudo apt purge nginx       # removes packaged configuration
```

Processes and signals

A process is a running program with a process ID, user, environment, open files, and resource usage. A port belongs to a listening socket owned by a process. Diagnose before killing.

```
ps aux
ps -ef --forest
top          # or htop when installed
pgrep -a nginx
sudo lsof -iTCP:80 -sTCP:LISTEN
sudo ss -lntup
kill -TERM <pid>      # ask for graceful shutdown
kill -KILL <pid>      # last resort; no cleanup
```

Foreground, background, and persistence

```
long-command &
jobs
fg %1
Ctrl+Z          # suspend foreground job
bg %1
nohup long-command > job.log 2>&1 &
```

nohup can keep a simple task alive after logout, but production services belong under systemd, a container restart policy, or an orchestrator. Those tools track exit state and restart behavior.

Lab: Trace a process to a port

Goal

Start a temporary HTTP server, find its PID and socket, test it, then stop it cleanly.

```
mkdir -p ~/labs/http
cd ~/labs/http
printf "hello from the VM\n" > index.html
python3 -m http.server 8080 > server.log 2>&1 &
SERVER_PID=$!
printf "PID=%s\n" "$SERVER_PID"
ss -lntp | grep ':8080'
curl -i http://127.0.0.1:8080/
kill -TERM "$SERVER_PID"
wait "$SERVER_PID"
ss -lntp | grep ':8080' || echo "port closed"
```

Evidence of success

- The HTTP response contains 200 OK and your text.
- The listening socket appears while the process runs and disappears after termination.
- You can distinguish process identity, port number, and URL.

3 Editing on a server: Nano and Vim

Outcome. Edit configuration safely in either editor and recover from the most common mode and save mistakes.

Choose the tool that preserves accuracy

Nano presents shortcuts on screen and works well for small edits. Vim uses modes and rewards practice with fast navigation and precise changes. Both appear on production servers. You need basic survival skills in each; you do not need to make either editor part of your identity.

Nano

```
nano notes.txt

Ctrl+O      write file (then Enter confirms the name)
Ctrl+X      exit
Ctrl+W      search
Alt+W       next match
Ctrl+K      cut current line
Ctrl+U      paste
Alt+U       undo
Alt+E       redo
Ctrl+G      help
```

Nano uses the caret symbol for Control, so ^O means Ctrl+O. A prefix such as M-U means Meta/Alt+U. If Alt shortcuts fail in a terminal, press Esc and then the letter.

Vim's modal model

Mode	Enter it	Purpose
Normal	Esc	move, delete, copy, paste, search, commands
Insert	i, a, o	type text
Visual	v or V	select characters or lines
Command-line	:	save, quit, substitute, configure

```
vim notes.txt

i          insert before cursor
Esc        return to Normal mode
:w         write (save)
:q         quit
:wq        save and quit
:q!        quit and discard unsaved changes

h j k l    move left, down, up, right
w / b      next / previous word
0 / $      line start / end
gg / G     first / last line
dd         delete line
yy         copy line
p          paste after cursor
u / Ctrl+r undo / redo
/listen    search forward; n repeats
:%s/old/new/g replace all occurrences
:set number show line numbers
```

If you feel trapped in Vim

Press Esc twice, type :q!, then press Enter. That exits without saving. Use :wq when you want to keep changes.

Edit configuration defensibly

- 1 Copy the file or rely on version control before changing it.

- 2 Open it with the least privilege that works. Use `sudoedit` for protected files.
- 3 Make one logical change.
- 4 Run the program's configuration test.
- 5 Reload rather than restart when supported.
- 6 Read status and logs, then test the user-facing behavior.

```
sudoedit /etc/nginx/sites-available/myapp
sudo nginx -t
sudo systemctl reload nginx
systemctl status nginx --no-pager
curl -I http://127.0.0.1/
```

Why `sudoedit`

`sudoedit` copies a protected file to a temporary location, opens it with your editor as your normal user, and writes it back with elevated privilege. This reduces the time an editor and its plugins run as root.

Lab: Make the same edit twice

Goal

Practice both editors on disposable files so an emergency server edit does not become your first attempt.

```
mkdir -p ~/labs/editors
printf "PORT=8000\nLOG_LEVEL=info\n" > ~/labs/editors/app.env
cp ~/labs/editors/app.env ~/labs/editors/app.nano.env
cp ~/labs/editors/app.env ~/labs/editors/app.vim.env
nano ~/labs/editors/app.nano.env
vim ~/labs/editors/app.vim.env
diff -u ~/labs/editors/app.nano.env ~/labs/editors/app.vim.env || true
```

Evidence of success

- Both files contain the intended value.
- You saved and exited each editor without killing the terminal.
- The diff is empty when you made identical edits.
- Run `vimtutor` and complete its first lessons.

4 Networking without hand-waving

Outcome. Trace a request through DNS, IP, ports, sockets, HTTP, and firewalls instead of guessing at connectivity.

The layers you need every day

Concept	Question it answers	Example
DNS	What IP address corresponds to this name?	app.example.com → 203.0.113.10
IP address	Which network interface or host?	192.168.56.20
Route	Where should the next packet go?	default via 192.168.56.1
TCP/UDP port	Which service on the host?	22 SSH, 80 HTTP, 443 HTTPS
Socket	Which process owns the endpoint?	nginx listens on 0.0.0.0:80
HTTP	What application request and response?	GET /health → 200 OK
Firewall	Should this packet cross this boundary?	allow TCP 443, deny database port

Addresses and bind scope

127.0.0.1 is loopback: only this host can reach it. 0.0.0.0 as a listening address means all IPv4 interfaces. A private address such as 192.168.x.x can be reachable on a LAN or virtual network. A public address can be routed across the Internet, subject to firewalls and NAT.

```
ip -brief address
ip route
hostname -I
ss -ltnup
curl -v http://127.0.0.1:8000/health
curl -v http://192.168.56.20:8000/health
```

A listening port is not proof of reachability

The process may bind only to loopback. A host firewall, cloud security group, hypervisor NAT rule, router, or DNS record may still block the path. Test at each boundary.

DNS and HTTP inspection

```
getent hosts example.com
dig example.com A
curl -I https://example.com
curl -v https://example.com/health
openssl s_client -connect example.com:443 -servername example.com </dev/null
```

An HTTP status code proves that an HTTP-speaking component answered. It does not prove that every dependency works. A health endpoint should test the minimum state needed for its declared purpose. Use separate liveness and readiness checks when an orchestrator needs to distinguish process survival from traffic readiness.

A boundary-first diagnosis

- 1 Resolve the name: `getent hosts` or `dig`.
- 2 Confirm the client has a route: `ip route`.
- 3 Test the target port: `curl -v` or `nc -vz`.
- 4 On the server, identify the listener: `ss -ltnup`.
- 5 Inspect the host and cloud firewall.
- 6 Read the edge proxy log and the application log.

7 Test the backend directly from the proxy host.

Common ports

Port	Protocol	Typical use	Public exposure
22	TCP	SSH administration	Restrict by source or private access when possible
53	UDP/TCP	DNS	Only for a DNS service
80	TCP	HTTP / ACME redirect or challenge	Often public
443	TCP/UDP	HTTPS / HTTP/3	Public for web services
5432	TCP	PostgreSQL	Usually private only
6379	TCP	Redis	Private only
8000	TCP	Application development port	Bind locally behind the proxy

Lab: Prove each hop

Goal

Compare loopback and VM-network access to a temporary service.

```
cd ~/labs/http
python3 -m http.server 8080 --bind 127.0.0.1 > loopback.log 2>&1 &
LOOP_PID=$!
curl -sS http://127.0.0.1:8080/
VM_IP=$(hostname -I | awk '{print $1}')
curl --max-time 2 "http://${VM_IP}:8080/" || echo "expected: loopback-only"
kill "$LOOP_PID"

python3 -m http.server 8080 --bind 0.0.0.0 > all-interfaces.log 2>&1 &
ALL_PID=$!
curl -sS "http://${VM_IP}:8080/"
ss -lntp | grep ':8080'
kill "$ALL_PID"
```

Evidence of success

- Loopback succeeds in both cases.
- The VM address fails for the loopback-only listener and succeeds for the all-interface listener, unless a firewall blocks it.
- The `ss` output shows the different bind addresses.

5 Virtual machines as a safe server lab

Outcome. Build an isolated Ubuntu Server environment, choose its network mode, snapshot it, and treat it like a cloud VM.

What a VM isolates

A virtual machine has virtual CPU, memory, disks, network devices, firmware, and its own operating system kernel. A hypervisor maps those virtual devices to host resources. This costs more memory and startup time than a container, but it gives a strong, comprehensible machine boundary.

Production clouds also give you VMs, though the interface changes. An EC2 instance, Azure VM, or Compute Engine instance is still a guest operating system with disks, network interfaces, metadata, identity, and firewall rules.

VM versus container

Dimension	Virtual machine	Application container
Kernel	Own guest kernel	Shares host kernel
Isolation boundary	Machine / hardware abstraction	Processes, namespaces, control groups
Startup	Seconds to minutes	Usually seconds or less
Image size	Often gigabytes	Often megabytes to hundreds of MB
Best fit	OS isolation, cloud host, incompatible kernels	Packaging and running services
Common mistake	Treating snapshots as backups	Treating writable container layers as durable data

Local hypervisor choices

- Hyper-V: integrated with supported Windows editions; virtual switches control networking.
- VirtualBox or VMware Workstation: approachable desktop hypervisors with NAT, host-only, and bridged modes.
- KVM/QEMU + libvirt: the standard Linux virtualization stack; **virsh** and **virt-manager** manage guests. [S7]
- Multipass: quick Ubuntu instances for disposable command-line labs.

Network modes

Mode	Reachability	Use in this course
NAT	Guest reaches Internet through host; inbound access may need port forwarding	Safe default for package installation
Host-only	Host and guest share a private network; no direct Internet by itself	Excellent for predictable SSH and web testing
Bridged / external switch	Guest appears as another LAN device	Useful, but exposes the lab to the local network
Internal	Guests talk only to each other	Multi-VM network labs

Recommended lab topology

Use one NAT adapter for updates and one host-only adapter for a stable host-to-VM address. If your hypervisor supports only one simple mode, NAT plus an SSH port-forward also works.

Suggested Ubuntu VM

Resource	Starting value	Reason
vCPU	2	Enough for packages and small container builds
RAM	4 GB	Comfortable for Docker, Nginx, and a small app
Disk	30-40 GB dynamically allocated	Room for images and logs
OS	Current Ubuntu Server LTS	Long support window and broad documentation
User	A named admin user with sudo	Avoid direct root login
OpenSSH	Install during setup	Practice remote administration immediately

Snapshots are checkpoints, not backups

Take a snapshot after base installation and another before firewall or SSH changes. A snapshot tied to one VM store can disappear with the host disk. A backup is a separate, tested copy stored in a different failure domain.

First-boot inventory

```
hostnamectl
lsblk -f
df -h
free -h
ip -brief address
ip route
systemctl --failed
sudo apt update
sudo apt upgrade
```

Lab: Prepare the Ubuntu server

Goal

Create a named lab identity, update packages, and capture a baseline you can compare after later modules.

```
sudo hostnamectl set-hostname prod-lab
sudo apt update
sudo apt upgrade
mkdir -p ~/lab-baseline
hostnamectl > ~/lab-baseline/hostnamectl.txt
ip -brief address > ~/lab-baseline/ip-address.txt
ip route > ~/lab-baseline/routes.txt
systemctl --failed > ~/lab-baseline/failed-units.txt
dpkg-query -W -f='${binary:Package}\t${Version}\n' > ~/lab-baseline/packages.tsv
tar -czf ~/prod-lab-baseline.tar.gz -C ~ lab-baseline
```

Evidence of success

- The prompt or `hostname` shows `prod-lab`.
- Package commands finish without an unresolved error.
- The archive lists correctly with `tar -tzf ~/prod-lab-baseline.tar.gz`.
- Take a hypervisor snapshot named `base-updated`.

6 SSH keys and remote administration

Outcome. Authenticate with a protected key, verify the server identity, simplify connections, transfer files, and harden access without locking yourself out.

What SSH protects

SSH creates an encrypted channel between a client and an SSH server. The server proves its identity with a host key. The user then authenticates with a key, password, hardware-backed credential, or another configured method. Host authentication and user authentication solve different problems.

Public and private keys

Your private key stays on the client and signs an authentication challenge. The server stores the public key in `~/.ssh/authorized_keys`. The server does not “match” the two files by comparing their text; it verifies a signature that only the private key holder can produce.

Key handling

Share the `.pub` file. Protect the private key with a passphrase, restrict its filesystem permissions, keep it out of Git and backups you do not control, and revoke its public key when the client is lost. OpenSSH and Ubuntu recommend Ed25519 for general use. [S4][S5]

Generate and install a key

```
# On your laptop/client
ssh-keygen -t ed25519 -a 64 -C "vishnu@prod-lab"

# Linux/macOS client: install the public key
ssh-copy-id labuser@192.168.56.20

# Connect and inspect the negotiated session
ssh -v labuser@192.168.56.20

# On the server
ls -ld ~/.ssh
ls -l ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

The first connection displays a host-key fingerprint. Verify it through a trusted path such as the VM console or cloud console before accepting it. The `known_hosts` entry protects you against silently connecting to a different machine later.

A useful client configuration

```
# ~/.ssh/config on your laptop
Host prod-lab
  HostName 192.168.56.20
  User labuser
  IdentityFile ~/.ssh/id_ed25519
  IdentitiesOnly yes
  ServerAliveInterval 30
  ServerAliveCountMax 3

# Then:
ssh prod-lab
scp report.txt prod-lab:/home/labuser/
rsync -av --progress ./project/ prod-lab:/home/labuser/project/
```

Agent and forwarding

An SSH agent caches decrypted private-key operations so you do not type the passphrase for every connection. Agent forwarding lets a remote host request signatures from your local agent. That is convenient but increases trust in the remote host. Prefer repository deploy keys, short-lived credentials, or HTTPS tokens for server-side pulls.

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
ssh-add -l
```

```
# Do not enable ForwardAgent globally.
# Add it only to a host block when you understand the risk.
```

File transfer and persistent work

Tool	Use	Example
scp	Simple file or recursive copy over SSH	scp -r ./site prod-lab:srv/
sftp	Interactive SSH file transfer	sftp prod-lab
rsync	Incremental copy with metadata and resume behavior	rsync -az --delete src/ host:dst/
tmux	Keep a terminal session alive across disconnects	tmux new -s deploy
SSH tunnel	Reach a private port through the SSH channel	ssh -L 9000:127.0.0.1:8000 prod-lab

Be precise with **rsync --delete**

A trailing slash changes source semantics, and **--delete** removes destination files absent from the source. Run with **--dry-run** first.

Harden without losing access

Keep the working SSH session open. Confirm that a second session succeeds with the key before you disable passwords. Put local policy in a configuration snippet, validate the effective configuration, reload SSH, and test again. Ubuntu recommends testing with `sshd -t` before reload. [S5]

```
# /etc/ssh/sshd_config.d/10-local-hardening.conf
PubkeyAuthentication yes
PasswordAuthentication no
PermitRootLogin no
AllowUsers labuser

sudo sshd -t
sudo sshd -T | grep -E 'passwordauthentication|permitrootlogin|pubkeyauthentication'
sudo systemctl reload ssh
sudo journalctl -fu ssh.service
```

Lockout prevention

Do not close the original session until a new connection succeeds. Confirm the cloud firewall or UFW permits SSH. Keep VM or cloud console access available as the recovery path.

Lab: Key-only SSH with a tunnel

Goal

Use a key to reach the VM and forward a private HTTP port to your laptop.

```
# On the VM
cd ~/labs/http
python3 -m http.server 8000 --bind 127.0.0.1

# On the laptop, in another terminal
ssh -L 9000:127.0.0.1:8000 prod-lab

# On the laptop, in a third terminal
curl -i http://127.0.0.1:9000/
```

Evidence of success

- The web server listens only on VM loopback.
- The laptop reaches it through `localhost:9000` while the SSH tunnel is open.
- Closing SSH removes the forwarded path.
- You can explain why no public application port was required.

7 Services, systemd, and logs

Outcome. Manage long-running programs, read unit state, inspect the journal, and write a minimal service definition.

Service lifecycle

A Linux service usually runs without an interactive terminal. systemd starts it, tracks its process, orders dependencies, applies restart policy, captures journal messages, and can start it during boot. The unit file declares desired process behavior; it does not prove application health.

Command	Meaning
systemctl status nginx	Current unit state plus recent log lines
systemctl start nginx	Start now
systemctl stop nginx	Stop now
systemctl restart nginx	Stop and start; causes an interruption
systemctl reload nginx	Ask the service to re-read configuration when supported
systemctl enable nginx	Arrange startup at boot; does not necessarily start now
systemctl disable nginx	Remove boot-start links; does not necessarily stop now
systemctl is-active nginx	Machine-friendly current state
systemctl is-enabled nginx	Machine-friendly boot policy
systemctl --failed	List failed units

Read the journal with a question

journalctl reads entries captured by systemd-journald. Filter by unit, boot, time, priority, or follow mode instead of scrolling the entire journal. [S6]

```
sudo journalctl -u nginx --since "15 minutes ago"
sudo journalctl -u nginx -n 100 --no-pager
sudo journalctl -fu nginx
sudo journalctl -p warning..alert -b
sudo journalctl -k -b
sudo journalctl --disk-usage
```

A minimal custom service

```
# /etc/systemd/system/lab-clock.service
[Unit]
Description=Lab clock writer
After=network.target

[Service]
Type=simple
User=labuser
Group=labuser
WorkingDirectory=/home/labuser/labs/service
ExecStart=/usr/bin/python3 /home/labuser/labs/service/clock.py
Restart=on-failure
RestartSec=3
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target

# Apply a new or changed unit file
sudo systemctl daemon-reload
sudo systemctl enable --now lab-clock.service
systemctl status lab-clock.service --no-pager
journalctl -u lab-clock.service -n 30 --no-pager
```

daemon-reload versus service reload

`systemctl daemon-reload` makes systemd re-read unit files. `systemctl reload nginx` asks Nginx to re-read its own configuration. They affect different layers.

Exit state is evidence

A failed unit has a reason: an invalid executable path, permission denial, missing environment, port collision, bad configuration, or dependency failure. Read `systemctl status`, then the unit's journal. Run the exact executable as the configured service user when the error remains unclear.

Lab: Create and repair a service

Goal

Run a Python loop under systemd, break its path on purpose, diagnose the failure, and repair it.

```
mkdir -p ~/labs/service
cat > ~/labs/service/clock.py <<'PY'
from datetime import datetime
from time import sleep

while True:
    print(datetime.now().isoformat(), flush=True)
    sleep(5)
PY

sudoedit /etc/systemd/system/lab-clock.service
sudo systemctl daemon-reload
sudo systemctl enable --now lab-clock.service
systemctl status lab-clock.service --no-pager
journalctl -u lab-clock.service -f
```

Evidence of success

- The service is active and emits a timestamp every five seconds.
- After changing `ExecStart` to a wrong path and restarting, the unit fails.
- Status and journal output identify the bad executable path.
- After repair and `daemon-reload`, the service returns to active.

8 Docker: images, containers, storage, and networks

Outcome. Run and inspect containers, explain where state lives, and debug the boundary between a container and its host.

The container mental model

A container is a host process launched with isolation and resource controls. An image supplies a read-only filesystem template plus metadata. Docker creates a writable container layer, attaches networks and mounts, then starts the configured process. When that process exits, the container stops.

Object	Purpose	Persistence
Image	Versioned build output used to create containers	Remains until removed
Container	One runtime instance of an image	Writable layer is disposable
Volume	Docker-managed durable data	Survives container replacement
Bind mount	A host path mounted into a container	Lives at the host path
Network	Connectivity and service-name discovery	Independent lifecycle
Registry	Stores and distributes images	Remote service

Daily commands

```
docker version
docker info
docker run --rm hello-world
docker image ls
docker container ls
docker container ls -a
docker logs -f <container>
docker exec -it <container> sh
docker inspect <container>
docker stats
docker stop <container>
docker rm <container>
```

Ports: publish is a host-level decision

```
# Host port 8080 forwards to container port 80 on all host interfaces
docker run --rm -p 8080:80 nginx

# Safer for a backend behind host-level Nginx:
docker run --rm -p 127.0.0.1:8080:80 nginx

# In another terminal
ss -lntp | grep ':8080'
curl -I http://127.0.0.1:8080/
```

EXPOSE 8000 in a Dockerfile documents an intended container port. It does not publish that port on the host. The `-p` flag or Compose ports mapping publishes it.

Storage choices

```
docker volume create pgdata
docker volume inspect pgdata
docker run --rm -v pgdata:/var/lib/postgresql/data postgres:17

# Bind mount a local directory read-only
docker run --rm -v "$PWD/site:/usr/share/nginx/html:ro" -p 8080:80 nginx
```

Data rule

If replacing a container destroys important data, the design is incomplete. Put mutable data in a volume, managed database, object store, or explicit host path and test restoration.

Container networks

Containers on the same user-defined network can reach one another by container or service name. Do not use localhost to reach a different container; localhost means the current container.

```
docker network create labnet
docker run -d --name web --network labnet nginx
docker run --rm --network labnet curlimages/curl:8.16.0 http://web/
docker rm -f web
docker network rm labnet
```

Debug from both sides

- 1 Is the container running? `docker ps -a`.
- 2 Why did the main process exit? `docker logs` and `docker inspect`.
- 3 Does the process listen inside? `docker exec` then inspect sockets or curl.
- 4 Is the port published on the host? `docker port` and `ss`.
- 5 Can another service resolve the container name? Test from the same Docker network.
- 6 Is a mount hiding files that exist in the image? Inspect `Mounts`.

Docker group warning

Membership in the `docker` group grants control equivalent to root on a typical Docker host. Treat it as privileged access; do not grant it casually.

Lab: Run, inspect, and replace a container

Goal

Serve a page from a named volume, prove that a replacement container sees the same data, and keep the port local.

```
docker volume create labsite
docker run --rm -v labsite:/site alpine sh -c 'printf "persistent hello\n" > /site/index.html'

docker run -d --name lab-nginx -p 127.0.0.1:8080:80 -v labsite:/usr/share/nginx/html:ro nginx:alpine
curl http://127.0.0.1:8080/
docker inspect lab-nginx
docker rm -f lab-nginx

docker run -d --name lab-nginx-2 -p 127.0.0.1:8080:80 -v labsite:/usr/share/nginx/html:ro nginx:alpine
curl http://127.0.0.1:8080/
```

Evidence of success

- Both containers return `persistent hello`.
- The host listener binds to 127.0.0.1, not every interface.
- Removing the first container does not remove the named volume.
- You can find the volume in `docker volume ls`.

9 Dockerfiles and Compose

Outcome. Build a small non-root application image, define a repeatable multi-service runtime, and understand what belongs in each file.

Dockerfile versus Compose

A Dockerfile describes how to build one image. A compose.yaml describes how services run together: images, ports, environment, volumes, networks, health checks, and restart policy. Docker's current CLI uses docker compose; the standalone docker-compose installation is legacy. [S8][S9]

A production-minded Python image

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1    PYTHONUNBUFFERED=1

WORKDIR /app

RUN groupadd --system app      && useradd --system --gid app --create-home app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY --chown=app:app app ./app
USER app

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Copy dependency metadata before source code so unchanged dependencies can reuse the build cache. Run as a non-root user, omit unnecessary packages, keep the image small, use a .dockerignore, and rebuild regularly. Docker's official guidance also recommends trusted base images, multi-stage builds where useful, and CI builds/tests. [S10]

.dockerignore

```
.git
.venv
__pycache__
*.pyc
.env
.pytest_cache
tests/
docs/
output/
```

Secrets do not belong in image layers

Do not use ARG, ENV, COPY, or a committed .env file for production secrets. Use the deployment platform's secret mechanism or BuildKit secret mounts for build-time credentials.

Compose for one application

```
services:
  app:
    build:
      context: .
    image: production-lab-app:1.0.0
    restart: unless-stopped
    ports:
      - "127.0.0.1:8000:8000"
    environment:
      APP_ENV: production
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health')"
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s
```

Operate the Compose project

```
docker compose config
docker compose build --pull
docker compose up -d
docker compose ps
docker compose logs -f app
docker compose exec app python --version
docker compose restart app
docker compose down

# Do not add -v unless you intend to delete named volumes.
docker compose down -v
```

`docker compose config` renders the resolved configuration and catches many YAML or variable mistakes before the runtime changes. Compose can support a single-server production deployment, but you still need production-specific settings, durable data, logging, monitoring, and deployment discipline. Docker documents production override files and controlled service recreation. [S8]

Tags, digests, and releases

A tag such as 1.0.0 is a human label and can be moved. A digest identifies image content. Production deployments should record the exact image digest or immutable release reference. Avoid latest when you need a reproducible rollback.

```
docker image inspect production-lab-app:1.0.0 --format '{{json .RepoDigests}}'
docker image inspect production-lab-app:1.0.0 --format '{{.Id}}'
```

Lab: Build a health-checked service

Goal

Build the capstone app image early, run it through Compose, and inspect its health and logs.

```
mkdir -p ~/production-lab/app
cd ~/production-lab
# Create app/main.py, requirements.txt, Dockerfile, .dockerignore, compose.yaml
# using the complete files in Module 13.

docker compose config
docker compose build --pull
docker compose up -d
docker compose ps
curl -i http://127.0.0.1:8000/health
docker compose logs --tail 50 app
```

Evidence of success

- Compose resolves without warnings about missing variables.
- The container becomes healthy.
- The health endpoint returns JSON with HTTP 200.
- The application port listens only on host loopback.

10 Nginx and the reverse proxy boundary

Outcome. Explain why direct application serving can work, identify when a reverse proxy earns its place, and configure the essential forwarding headers.

Can an app run without Nginx?

Yes. A framework or application server can listen on a network socket and serve HTTP directly. That is common in development, internal services, and platforms where a managed load balancer or ingress proxy already supplies the edge boundary.

On a self-managed VM, Nginx often becomes the public entry point because it can terminate TLS, route by host or path, serve static content, enforce request limits, buffer slow clients, and proxy to one or more private backends. Your application then concentrates on application behavior.

Forward proxy versus reverse proxy

Type	Represents	Client knows	Typical use
Forward proxy	Clients	The destination Internet service	Organization egress policy, privacy, filtering
Reverse proxy	Servers	The public site; backend details stay internal	TLS, routing, balancing, edge controls

Request flow

- 1 The client resolves the domain to the server or load balancer address.
- 2 The client opens TCP port 80 or 443 and, for HTTPS, completes a TLS handshake.
- 3 Nginx selects a server block using the listening socket and host name.
- 4 A location block selects handling for the request path.
- 5 `proxy_pass` opens or reuses a connection to the backend.
- 6 The application response returns through Nginx to the client.

Minimal reverse proxy

```
server {
    listen 80;
    listen [::]:80;
    server_name app.example.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_connect_timeout 5s;
        proxy_read_timeout 60s;
    }
}
```

Nginx's proxy module forwards requests to another server with `proxy_pass`. The official example also sets the original host and client address headers. [S11] Your application must trust forwarded headers only from known proxies; otherwise a direct client can forge them.

Enable, validate, reload, verify

```
sudo ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/myapp
sudo nginx -t
sudo systemctl reload nginx
systemctl status nginx --no-pager
curl -i -H 'Host: app.example.com' http://127.0.0.1/
sudo tail -f /var/log/nginx/access.log /var/log/nginx/error.log
```

Static files and uploads

```
location /static/ {
    alias /srv/myapp/static/;
    expires 7d;
    add_header Cache-Control "public";
}

client_max_body_size 10m;
```

Choose upload limits from product requirements. Nginx can reject oversized bodies before they consume application workers. Keep user uploads outside the application image and treat uploaded files as untrusted content.

Load balancing is one capability, not magic scaling

```
upstream app_pool {
    least_conn;
    server 127.0.0.1:8001;
    server 127.0.0.1:8002;
}

location / {
    proxy_pass http://app_pool;
}
```

Two application processes help only if the database, sessions, cache, jobs, and downstream limits can support them. Store shared state outside an individual process before scaling horizontally.

Lab: Put Nginx in front of a private backend

Goal

Proxy host port 80 to an app bound only on 127.0.0.1:8000, then prove each endpoint separately.

```
sudo apt update
sudo apt install nginx
curl -i http://127.0.0.1:8000/health
sudoedit /etc/nginx/sites-available/production-lab
sudo ln -s /etc/nginx/sites-available/production-lab /etc/nginx/sites-enabled/production-lab
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
curl -i http://127.0.0.1/health
ss -lntp | grep -E ':80|:8000'
```

Evidence of success

- Direct backend health works on 127.0.0.1:8000.
- Nginx health works on port 80.
- Port 80 listens publicly as intended; port 8000 listens only on loopback.
- The Nginx access log records the request and the application log records backend handling.

11 TLS, firewalls, secrets, and server hardening

Outcome. Apply layered controls while preserving a recovery path and distinguish encryption, authentication, authorization, and exposure.

Four different security questions

Control	Question	Example
Authentication	Who or what is making the request?	SSH key, user login, workload identity
Authorization	What may that identity do?	sudo policy, file mode, application role
Encryption	Can observers read or alter data in transit?	SSH and TLS
Network policy	Which endpoints may communicate?	UFW, security groups, private subnets

Host firewall with UFW

UFW provides a simpler interface for host-level firewall rules. It starts disabled on a default Ubuntu installation. Permit your management path before enabling it. Ubuntu documents status, numbered rules, and source-specific SSH rules. [S12]

```
sudo ufw status verbose
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
sudo ufw status numbered

# Tighter SSH example when the admin source is stable:
sudo ufw allow from 192.168.56.1 to any port 22 proto tcp
```

Cloud firewall plus host firewall

A cloud security group and UFW both need to allow intended traffic. Use the cloud console as a recovery path and add rules in an order that preserves SSH access.

HTTPS and certificates

TLS encrypts the HTTP channel and lets the client validate the server name through a certificate chain. Let's Encrypt is an automated public certificate authority, and Certbot can obtain and install certificates for Nginx. [S13][S14] A public certificate requires control of the domain and successful ACME validation; an IP address alone does not follow the same ordinary web flow.

```
# After DNS points app.example.com to the VM and ports 80/443 work:
sudo apt update
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d app.example.com
sudo certbot renew --dry-run
systemctl status certbot.timer --no-pager

# Verify from another machine:
curl -I https://app.example.com
openssl s_client -connect app.example.com:443 -servername app.example.com </dev/null
```

Certificate automation changes configuration

Back up or version the Nginx configuration and run `nginx -t` after certificate tooling edits it. Certbot's documentation recommends backing up Nginx configuration. [S15]

Secrets

- Commit a `.env.example` with variable names and safe placeholders, not a real `.env`.
- Grant secrets only to the workload identity that needs them.

- Prefer a cloud or platform secret manager for production rotation and audit.
- Redact headers, tokens, passwords, private keys, and connection strings from logs.
- Rotate a secret after suspected exposure; deleting it from the latest Git commit does not erase history.

Patch, minimize, and review

```
sudo apt update
apt list --upgradable
sudo unattended-upgrade --dry-run --debug
systemctl --failed
sudo ss -lntup
sudo journalctl -p warning..alert -b
docker image ls
docker compose ps
```

Install only required software, disable unused services, remove public ports you do not need, rebuild images with patched dependencies, and record change windows. Security work without ownership and a maintenance routine decays.

Backups require restore tests

Asset	Backup method	Restore proof
Database	Native consistent dump or managed snapshot	Restore to an isolated instance and query it
Uploaded files	Versioned object storage or filesystem backup	Recover known sample and verify checksum
Configuration	Git plus protected secret backup	Rebuild a clean VM from documentation
Container images	Registry with immutable release references	Deploy previous digest
TLS/account keys	Provider-specific secure recovery	Document renewal and revocation path

Lab: Reduce the exposed surface

Goal

Enable a host firewall only after preserving SSH, then verify intended and unintended ports.

```
sudo ufw status verbose
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
sudo ufw status numbered
ss -lntup

# From the laptop
ssh prod-lab
curl -I http://192.168.56.20/
curl --max-time 2 http://192.168.56.20:8000/ || echo "expected: private backend"
```

Evidence of success

- A fresh SSH connection still succeeds.
- Nginx responds on port 80.
- The backend port is unavailable from the laptop because it binds to VM loopback.
- UFW lists only the rules you intended.

12 Git, deployment, observability, and incident habits

Outcome. Turn server commands into a repeatable engineering workflow with reviewable changes, explicit releases, checks, and rollback.

Git is history, not deployment

Git records source and configuration changes. A deployment selects a reviewed commit, builds or fetches an artifact, applies configuration, starts it, verifies behavior, and records the release. Pulling the latest branch on a server is easy, but it hides build drift and makes rollback ambiguous.

```
git status
git switch -c feature/health-endpoint
git diff
git add app/main.py tests/test_health.py
git diff --cached
git commit -m "Add health endpoint"
git log --oneline --decorate -n 10
git fetch origin
git pull --ff-only
git push -u origin feature/health-endpoint
```

Git's pull combines fetch with an integration choice. Teams should choose and document merge, rebase, or fast-forward policy instead of relying on accidental defaults. [S16]

Configuration as code

- Keep Dockerfiles, Compose files, Nginx templates, service units, database migrations, and runbooks under review.
- Parameterize environment differences; do not fork production configuration into undocumented snowflakes.
- Validate configuration in CI: tests, linters, **docker compose config**, and build checks.
- Keep secrets outside source control and inject them through the deployment environment.

A small release protocol

- 1 Identify the exact release commit and image digest.
- 2 Confirm tests, image build, and vulnerability policy completed.
- 3 Back up mutable data when the change can affect it.
- 4 Read the migration and rollback plan.
- 5 Deploy one controlled change.
- 6 Verify process state, health endpoint, external request path, and logs.
- 7 Observe for a defined window and record the result.
- 8 Roll back when the agreed threshold fails; investigate after service is stable.

Logs, metrics, and traces answer different questions

Signal	Best question	Example
Logs	What happened in this request or process?	exception, deploy event, auth failure
Metrics	How is behavior changing over time?	request rate, latency percentile, error rate
Traces	Where did one distributed request spend time?	proxy → API → database span
Health checks	Can the platform send work now?	readiness endpoint
Synthetic checks	Can an external user complete a critical path?	login or API probe

Minimum useful application log fields

Use a timestamp, severity, service name, release version, environment, request or correlation ID, event name, and safe structured context. Do not log passwords, bearer tokens, session cookies, private keys, or complete sensitive request bodies.

Incident loop

- 1 Stabilize: stop the harm or restore service with the safest known action.
- 2 Scope: identify affected users, time window, versions, and dependencies.
- 3 Preserve: retain logs, release metadata, and relevant state.
- 4 Diagnose: form a testable hypothesis and gather evidence at each boundary.
- 5 Repair: make the smallest safe change and verify it.
- 6 Learn: update tests, alerts, runbooks, ownership, and design.

Rollback is a feature

A rollback plan must name the prior artifact, configuration compatibility, database implications, commands, owner, and verification checks. “Redeploy the old version” is incomplete.

Automation progression

Stage	Appropriate practice
Learning VM	Manual commands plus a written checklist
Repeatable single VM	Compose, systemd, versioned Nginx config, scripted checks
Team service	CI build/test, registry, secret manager, automated deployment, monitoring
Scaled platform	Orchestrator or managed runtime, immutable artifacts, policy, centralized telemetry

Lab: Write and rehearse a release checklist

Goal

Deploy a harmless version change and prove that you can return to the previous image.

```
cd ~/production-lab
git status
git rev-parse --short HEAD
docker compose config
docker compose build --pull
docker image tag production-lab-app:1.0.0 production-lab-app:1.1.0
```

Update the Compose image reference, then:

```
docker compose up -d
docker compose ps
curl -fsS http://127.0.0.1/health
docker compose logs --since 5m
```

Rollback rehearsal: restore the 1.0.0 reference and:

```
docker compose up -d
curl -fsS http://127.0.0.1/health
```

Evidence of success

- You recorded the release commit or local lab revision.
- The deployment check includes Nginx and the backend, not container state alone.
- The previous image remains available and starts successfully.
- Your notebook records time, commands, evidence, and rollback result.

13 Capstone: deploy a FastAPI service on the VM

Outcome. Assemble the full chain: SSH administration, Git-ready files, a non-root Docker image, Compose health checks, host Nginx, firewall policy, logs, and rollback evidence.

Target architecture



Browser path client → DNS → VM public IP → Nginx → private app port

Admin path terminal → SSH :22 → shell → systemctl / docker / logs

The database can live in another container or a managed service. Do not expose it to the public Internet.

Figure 2. Nginx exposes the service; the container port remains on VM loopback.

This pattern suits a small self-managed VM and teaches the boundaries clearly. At greater scale, a managed load balancer or ingress controller may replace host Nginx, and a registry supplies the image.

Pre-flight checklist

- Ubuntu VM is updated and has at least 2 vCPU, 4 GB RAM, and sufficient disk space.
- SSH key login works in a second terminal.
- Docker Engine and the Compose plugin are installed from a trusted supported source.
- Nginx is installed on the VM.
- UFW permits SSH before it is enabled.
- You have a VM snapshot or console recovery path.

```
docker version
docker compose version
nginx -v
python3 --version
df -h
free -h
systemctl --failed
```

1. Create the project layout

```
mkdir -p ~/production-lab/app
cd ~/production-lab
```

```
production-lab/
├── app/
│   └── main.py
├── .dockerignore
├── .env.example
├── compose.yaml
├── Dockerfile
├── README.md
└── requirements.txt
```

Use Nano, Vim, or an SSH-aware editor to create the following files. Keep the project in Git. Do not commit real secret values.

2. Application

app/main.py

```
from __future__ import annotations

import os
import socket
from datetime import datetime
from datetime import timezone

from fastapi import FastAPI

APP_VERSION = os.getenv("APP_VERSION", "dev")

app = FastAPI(title="Production Lab", version=APP_VERSION)


@app.get("/")
async def root() -> dict[str, str]:
    return {
        "message": "Production lab is running",
        "version": APP_VERSION,
        "host": socket.gethostname(),
    }


@app.get("/health")
async def health() -> dict[str, str]:
    return {
        "status": "ok",
        "version": APP_VERSION,
        "time": datetime.now(timezone.utc).isoformat(),
    }
```

requirements.txt

```
fastapi==0.116.1
uvicorn[standard]==0.35.0
```

Dependency note

These versions make the lab reproducible as written. Before a real deployment, review current supported versions, security advisories, and compatibility; then update and test deliberately.

3. Image definition

Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1    PYTHONUNBUFFERED=1

WORKDIR /app

RUN groupadd --system app      && useradd --system --gid app --create-home app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY --chown=app:app app ./app

USER app
EXPOSE 8000

CMD [
    "uvicorn",
    "app.main:app",
    "--host",
    "0.0.0.0",
    "--port",
    "8000",
    "--proxy-headers",
    "--forwarded-allow-ips="
]
```

The wildcard proxy trust is acceptable in this lab only because Compose publishes the backend on VM loopback and Nginx is the only intended caller. On a shared network, specify the trusted proxy address or subnet instead.

.dockerignore

```
.git
.env
.venv
__pycache__
*.pyc
.pytest_cache
output/
tmp/
```

.env.example

```
APP_VERSION=1.0.0
```

4. Runtime definition

compose.yaml

```
services:
  app:
    build:
      context: .
    image: production-lab-app:1.0.0
    restart: unless-stopped
    ports:
      - "127.0.0.1:8000:8000"
    environment:
      APP_VERSION: "1.0.0"
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health')"
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s
    read_only: true
    tmpfs:
      - /tmp:size=64m,mode=1777
    security_opt:
      - no-new-privileges:true
```

5. Validate and start the app

```
cd ~/production-lab
docker compose config
docker compose build --pull
docker compose up -d
docker compose ps
docker compose logs --tail 100 app
curl -i http://127.0.0.1:8000/
curl -i http://127.0.0.1:8000/health
ss -lntp | grep ':8000'
```

Stop here if health fails

Do not add Nginx until the backend works on VM loopback. Read the container status, health output, and logs. Adding another layer to a broken backend makes diagnosis slower.

6. Configure host Nginx

/etc/nginx/sites-available/production-lab

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;

    access_log /var/log/nginx/production-lab.access.log;
    error_log /var/log/nginx/production-lab.error.log;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_connect_timeout 5s;
        proxy_read_timeout 60s;
    }
}
```

```

sudo ln -s /etc/nginx/sites-available/production-lab /etc/nginx/sites-enabled/production-lab
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
systemctl status nginx --no-pager
curl -i http://127.0.0.1/
curl -i http://127.0.0.1/health

```

7. Apply and verify firewall policy

```

sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
sudo ufw status verbose

# On the laptop; replace the address with the VM host-only/LAN address.
curl -i http://192.168.56.20/
curl --max-time 2 http://192.168.56.20:8000/ || echo "expected: backend is not public"
ssh prod-lab

```

8. Observe the full request

```

# Terminal A: Nginx logs
sudo tail -f /var/log/nginx/production-lab.access.log /var/log/nginx/production-lab.error.log

# Terminal B: application logs
cd ~/production-lab
docker compose logs -f app

# Laptop or Terminal C: generate requests
curl -i http://192.168.56.20/
curl -i http://192.168.56.20/health

```

One request should appear first at Nginx and then at Uvicorn. Stop the app container and repeat the request. Nginx should return a gateway error and record a connection failure. Start the app and verify recovery. This controlled break teaches the signature of an unavailable upstream.

9. Optional domain and HTTPS

- 1 Create a DNS A/AAAA record for your domain pointing to the reachable server address.
- 2 Replace `server_name _;` with the real domain and validate Nginx.
- 3 Permit ports 80 and 443 at every firewall boundary.
- 4 Use the current Certbot installation instructions for your OS.
- 5 Run Certbot for the domain and then `certbot renew --dry-run`.
- 6 Verify certificate name, chain, redirect behavior, and automatic renewal timer.

10. Reboot and rollback tests

```

sudo reboot

# After reconnecting:
systemctl is-active docker nginx
cd ~/production-lab
docker compose ps
curl -fsS http://127.0.0.1/health

# Rollback pattern:
# 1. restore previous compose.yaml / image tag or digest
# 2. validate the resolved configuration
docker compose config
docker compose up -d
curl -fsS http://127.0.0.1/health

```

Capstone acceptance checklist

Check	Command or evidence	Pass condition
SSH	New key-only session	Login succeeds; private key remains on client
Image	docker compose build	Build completes; process runs as non-root
Backend	curl 127.0.0.1:8000/health	HTTP 200 with expected version
Health	docker compose ps	Service reports healthy
Proxy	curl 127.0.0.1/health	Nginx returns backend response
Exposure	ss + remote curl	80/443 public as intended; 8000 loopback only
Logs	Nginx + Compose logs	One request traceable across both layers
Boot	Reboot test	Docker, Nginx, and app recover
Rollback	Previous image/config	Prior release restores and passes health

14 Troubleshooting playbook

Outcome. Diagnose by layer using observable evidence, recognize frequent failure signatures, and avoid high-noise fixes.

The golden sequence

1. What exact command or request failed?
2. What was the exit code, HTTP status, or error text?
3. What changed immediately before failure?
4. Which boundary is the last one proven healthy?
5. What single observation would falsify the current hypothesis?
6. What is the smallest reversible test?

Fast triage

```
date -Is
uptime
df -h
free -h
systemctl --failed
ss -lntup
systemctl status nginx --no-pager
sudo nginx -t
docker compose ps
docker compose logs --tail 100
curl -v http://127.0.0.1:8000/health
curl -v http://127.0.0.1/health
```

Failure matrix

Symptom	Likely layer	Evidence and next action
SSH timeout	Route / firewall / sshd	Verify IP and VM network; test port 22; check UFW/cloud rules and systemctl status ssh from console.
SSH permission denied (publickey)	User/key policy	Use ssh -vvv; verify user, IdentityFile, authorized_keys, modes, AllowUsers, and SSH journal.
REMOTE HOST IDENTIFICATION HAS CHANGED	Host identity	Do not delete blindly. Confirm the VM was rebuilt or host key rotated through a trusted console, then update the exact known_hosts entry.
Nginx 502 Bad Gateway	Proxy → backend	Read Nginx error log; curl backend on 127.0.0.1:8000; inspect app container, port, and proxy_pass.
Nginx config test fails	Nginx configuration	Read file and line number from nginx -t; fix syntax before reload.
Container exits immediately	Main process / image	docker compose ps -a, logs, inspect exit code; run the command locally or override entrypoint for diagnosis.
Container healthy, remote access fails	Port publication / firewall	Check Compose ports, host ss, bind address, UFW, hypervisor NAT, and cloud rules.
Works by IP, not domain	DNS / virtual host / certificate	Resolve A/AAAA, curl with Host header, inspect Nginx server_name, validate TLS SNI and certificate name.
No space left on device	Disk / inode exhaustion	df -h, df -i, du, journal usage, Docker disk usage. Preserve evidence before cleanup.
Permission denied writing a mount	UID/GID / filesystem mode	Inspect process user, mount owner, parent-directory modes, read-only flag, and security confinement logs.

Docker-specific evidence

```
docker compose config
docker compose ps -a
docker compose logs --timestamps --tail 200 app
docker inspect production-lab-app-1
docker inspect --format '{{json .State.Health}}' production-lab-app-1
docker system df
docker events --since 10m
docker network inspect production-lab_default
```

Nginx-specific evidence

```
sudo nginx -t
sudo nginx -T | less
systemctl status nginx --no-pager
sudo journalctl -u nginx --since "15 minutes ago"
sudo tail -n 100 /var/log/nginx/production-lab.error.log
curl -v -H 'Host: app.example.com' http://127.0.0.1/
curl -v http://127.0.0.1:8000/health
```

High-noise actions to resist

- Rebooting before collecting logs and state.
- Deleting containers, volumes, caches, or package state without identifying the failure.
- Changing several firewall, proxy, and application settings together.
- Using `chmod 777` or running the application as root to bypass a permission problem.
- Disabling TLS verification, host-key checks, or authentication to “see if it works.”
- Rebuilding with unpinned dependencies when you need to reproduce the current failure.

Evidence notebook

Record timestamps, exact commands, relevant output, versions, and what changed. A clean incident timeline often reveals the cause before another configuration change does.

Practice fault injection

Break	Expected symptom	Repair
Stop the app container	Nginx 502; connection refused in error log	Start Compose service; verify health
Use wrong proxy port	Nginx 502 while backend works directly	Restore <code>proxy_pass</code> ; <code>nginx -t</code> ; reload
Create invalid Nginx directive	<code>nginx -t</code> fails with file and line	Undo edit; retest; do not reload broken config
Bind app to wrong internal address	Health check or published port fails	Listen on 0.0.0.0 inside container
Fill a disposable filesystem	Writes fail; service may crash	Identify consumer; clean safely; add alert/retention

15 Interview explanations and a 30-day practice plan

Outcome. Explain the architecture in your own words and convert the material into repeated operating skill.

Short explanations that preserve the important detail

What is SSH?

SSH is an encrypted protocol for remote login, command execution, tunneling, and file transfer. The server authenticates itself with a host key. The user then authenticates, often by proving possession of a private key whose public counterpart the server authorizes.

Why use keys instead of passwords?

A strong private key is not guessable through ordinary online password attempts, and the server does not need the private key. Keys can be unique per device, revoked individually, protected by a passphrase or hardware, and constrained in `authorized_keys`. Key authentication still needs secure storage, host verification, and lifecycle management.

What is a reverse proxy?

A reverse proxy is the server-side entry point that accepts client requests, selects a backend, forwards the request, and returns the backend response. It can centralize TLS, routing, request limits, static content, buffering, and load balancing while keeping application ports private.

Why Nginx if FastAPI can serve HTTP?

FastAPI through an ASGI server can serve HTTP directly, and managed platforms may already provide an edge proxy. On a self-managed VM, Nginx commonly handles public sockets, TLS, host/path routing, static files, and edge controls. The application server remains a private backend.

Container versus VM

A VM virtualizes a machine and runs a guest kernel. A container packages a process and user-space filesystem while sharing the host kernel. VMs give a strong OS boundary and can run different kernels; containers start faster and package application dependencies efficiently.

Dockerfile versus Compose

A Dockerfile builds an image. A Compose file defines how one or more services run: images, build contexts, environment, ports, volumes, networks, dependencies, health checks, and restart policies.

What does systemd do?

systemd manages the lifecycle and ordering of Linux services and other units. It starts programs, tracks their state, applies restart and dependency rules, integrates logs with the journal, and can enable services at boot.

What happens when you open `https://app.example.com`?

DNS resolves the name. The client routes packets to the address and opens a connection on port 443. TLS authenticates the server name and encrypts the channel. Nginx selects the virtual host, forwards the HTTP request to the private application, and returns the response. Each step can fail independently.

Interview depth ladder

Level	Answer quality
Definition	Name the tool or concept accurately.
Mechanism	Explain the request, process, storage, or authentication path.
Trade-off	State when the tool helps and when another layer already provides the function.
Failure	Name evidence and a realistic failure signature.
Operations	Describe validation, monitoring, rollback, and security implications.

30-day plan

Days	Focus	Deliverable
1-3	Shell, paths, redirection, search	VM inventory folder and command notebook
4-5	Users, permissions, processes	Port-to-process lab and permission explanation
6-7	Nano, Vim, manuals	Same config edit in both editors; complete vimtutor basics
8-10	Networking, curl, DNS, sockets	Boundary map for loopback, VM IP, and public path
11-13	VM networking and snapshots	Rebuildable Ubuntu VM baseline
14-16	SSH keys, config, copy, tunnel, tmux	Key-only access and private-port tunnel
17-18	systemd and journal	Custom service, deliberate failure, repair notes
19-22	Docker and Compose	Health-checked non-root app image
23-25	Nginx and reverse proxy	Backend private on loopback; Nginx public
26-27	UFW, TLS concepts, secrets, backups	Exposure audit and recovery checklist
28-29	Capstone and fault injection	Full deployment plus three repaired failures
30	Rebuild from notes	Fresh VM deployment without copying shell history

Portfolio evidence

- A diagram showing admin and user request paths.
- A repository with Dockerfile, Compose, Nginx template, README, and safe environment example.
- A runbook with deploy, verify, rollback, backup, and restore steps.
- Screenshots or terminal captures of healthy state and repaired fault injection.
- A short postmortem describing one deliberate failure, evidence, cause, fix, and prevention.

The strongest demonstration

Rebuild the stack on a fresh VM from your own repository and runbook. A working old VM proves the machine works. A successful clean rebuild proves you understand and documented the system.

A Command cheat sheets

Outcome. Keep a compact reference for the tasks you will repeat while the deeper explanations remain in earlier modules.

Linux navigation and files

Task	Command
Location / identity	pwd • whoami • hostname • id
List	ls -lah • find . -maxdepth 2 -type f
Navigate	cd /path • cd .. • cd ~ • cd -
Inspect	cat • less • head • tail -f • file • stat
Search	grep -Rni 'text' path • find path -name '*.conf'
Size	df -h • df -i • du -sh path
Copy / move	cp -a source dest • mv source dest
Archive	tar -czf archive.tar.gz directory • tar -tzf archive.tar.gz
Manuals	man command • command --help • apropos keyword

Nano and Vim

Tool	Essential actions
Nano	Ctrl+O save • Ctrl+X exit • Ctrl+W search • Ctrl+K cut line • Ctrl+U paste • Ctrl+G help
Vim	i insert • Esc normal • :w save • :q quit • :wq save/quit • :q! discard • / search • dd delete line • u undo

Networking

Question	Command
Addresses and routes	ip -brief address • ip route
Listening sockets	ss -lntup
DNS	getent hosts name • dig name A
HTTP	curl -i URL • curl -v URL • curl -fs URL
TLS	openssl s_client -connect host:443 -servername host
Port owner	sudo lsof -iTCP:port -sTCP:LISTEN
Firewall	sudo ufw status verbose • sudo ufw status numbered

SSH

Task	Command
Generate	ssh-keygen -t ed25519 -a 64 -C 'label'
Install public key	ssh-copy-id user@host
Connect	ssh user@host • ssh -v alias
Copy	scp file alias:/path/ • rsync -av --dry-run src/ alias:dst/
Tunnel	ssh -L 9000:127.0.0.1:8000 alias
Agent	eval "\$(ssh-agent -s)" • ssh-add ~/.ssh/id_ed25519
Server log	sudo journalctl -fu ssh.service
Validate policy	sudo sshd -t • sudo sshd -T

systemd and logs

Task	Command
State	systemctl status unit --no-pager • systemctl --failed
Lifecycle	start • stop • restart • reload
Boot policy	enable • disable • enable --now
New unit file	sudo systemctl daemon-reload
Logs	journalctl -u unit -n 100 --no-pager
Follow	journalctl -fu unit
This boot warnings	journalctl -p warning..alert -b

Docker and Compose

Task	Command
Inventory	docker image ls • docker ps -a • docker volume ls • docker network ls
Logs / shell	docker logs -f name • docker exec -it name sh
Inspect	docker inspect name • docker stats • docker system df
Compose validate	docker compose config
Build / start	docker compose build --pull • docker compose up -d
State / logs	docker compose ps • docker compose logs -f app
Stop	docker compose down
Delete volumes too	docker compose down -v (destructive to named project volumes)

Nginx

Task	Command
Validate	sudo nginx -t
Expanded config	sudo nginx -T less
Reload	sudo systemctl reload nginx
State	systemctl status nginx --no-pager
Journal	sudo journalctl -u nginx --since '15 minutes ago'
Files	/etc/nginx/sites-available • /etc/nginx/sites-enabled
Logs	/var/log/nginx/access.log • /var/log/nginx/error.log

B Glossary and sources

Outcome. Use precise language and continue learning from primary project documentation.

Glossary

Term	Meaning
Bind address	Local interface address on which a process accepts network connections.
Container image	Immutable filesystem and metadata template used to create containers.
Daemon	Long-running background process that provides a service.
DNS	System that maps domain names to records such as IP addresses.
Exit code	Integer status returned by a process; zero conventionally means success.
Firewall	Policy enforcement that allows or denies network traffic.
Host key	SSH server identity key used by clients to detect the wrong or changed server.
HTTP	Application protocol for request/response communication on the web.
Idempotent	An operation that can be repeated without adding unintended changes after the desired state is reached.
Least privilege	Grant only the access required for a defined task and duration.
Loopback	Network interface used to reach the same host, commonly 127.0.0.1.
Port	Numeric transport endpoint used to select a service on an IP address.
Private key	Secret key material used to prove identity or decrypt/sign; it must remain protected.
Public key	Shareable key material used to verify a signature or encrypt for the key holder.
Reverse proxy	Server-side intermediary that receives requests and forwards them to backends.
Socket	Operating-system endpoint for local or network communication.
TLS	Protocol that authenticates endpoints and encrypts network traffic, used by HTTPS.
Volume	Storage mounted separately from a container's disposable writable layer.

Primary sources

Tool behavior and installation steps can change. Use these upstream sources, your distribution manuals, and the exact version installed on your machine. Links were checked while preparing this July 2026 edition.

ID	Source
S1	GNU nano documentation https://www.nano-editor.org/docs.php
S2	Vim help files and getting started material https://vimhelp.org/
S3	Git reference and user manual https://git-scm.com/docs
S4	OpenBSD manual: ssh-keygen(1) https://man.openbsd.org/ssh-keygen.1
S5	Ubuntu Server: OpenSSH server https://documentation.ubuntu.com/server/how-to/security/openssh-server/
S6	systemd journalctl manual https://www.freedesktop.org/software/systemd/man/latest/journalctl.html
S7	Ubuntu Server: introduction to virtualization and libvirt https://ubuntu.com/server/docs/explanation/intro-to/virtualisation/

ID	Source
S8	Docker Docs: use Compose in production https://docs.docker.com/compose/how-tos/production/
S9	Docker Docs: Compose installation overview https://docs.docker.com/compose/install/
S10	Docker Docs: Dockerfile building best practices https://docs.docker.com/build/building/best-practices/
S11	Nginx: HTTP proxy module https://nginx.org/en/docs/http/nginx_http_proxy_module.html
S12	Ubuntu Server: firewall and UFW https://documentation.ubuntu.com/server/how-to/security/firewalls/
S13	Let's Encrypt documentation https://letsencrypt.org/docs/
S14	Certbot introduction https://eff-certbot.readthedocs.io/en/stable/intro.html
S15	Certbot user guide https://eff-certbot.readthedocs.io/en/stable/using.html
S16	Git pull documentation https://git-scm.com/docs/git-pull

Suggested next subjects

- Shell scripting with strict error handling and ShellCheck.
- Cloud-init, infrastructure as code, and configuration management.
- PostgreSQL administration, migrations, connection pools, backups, and restore drills.
- CI/CD artifact provenance, dependency scanning, image signing, and policy.
- Metrics with Prometheus/OpenTelemetry and structured centralized logging.
- Managed containers and Kubernetes after single-host networking and storage feel concrete.
- WireGuard, bastion hosts, private subnets, and zero-trust administration.

Keep the model

New tools will rename the controls, but the questions remain: which process, identity, file, socket, route, policy, dependency, artifact, log, and rollback path?